

Prácticas de Visión por Computador

Fundamentos de Python para manipulación de imágenes

Pablo Mesejo

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



- En estas diapositivas vemos una introducción a la lectura, visualización y manipulación de imágenes.
- Estas explicaciones se complementan con un Notebook que contiene estas instrucciones ejecutadas y más cosas (**Guia_fundamentos_imagenes.ipynb**).

Lectura de Imágenes

```
filename = "images/orapple.jpg"
```

```
import matplotlib.image as mpimg  
image1 = mpimg.imread(filename)
```

```
from PIL import Image  
image2 = Image.open(filename)
```

```
import cv2 as cv  
image3 = cv.imread(filename)
```



Python Imaging Library (PIL)



Lectura de Imágenes (y 2)

```
image1.shape
```

```
Out[1]: (535, 500, 3)
```



```
image2.shape
```

```
AttributeError: shape
```

```
import numpy as np
```

```
im = np.asarray(image2)
```

```
im.shape
```

```
Out[1]: (535, 500, 3)
```

Python Imaging Library (PIL)

```
image3.shape
```

```
Out[1]: (535, 500, 3)
```



Visualización de Imágenes

```
from IPython.display import Image  
Image(filename)
```

IP[y]: IPython
Interactive Computing

```
import matplotlib.pyplot as plt  
plt.imshow(image1)
```

matplotlib

```
display(image2)
```

Python Imaging Library (PIL)

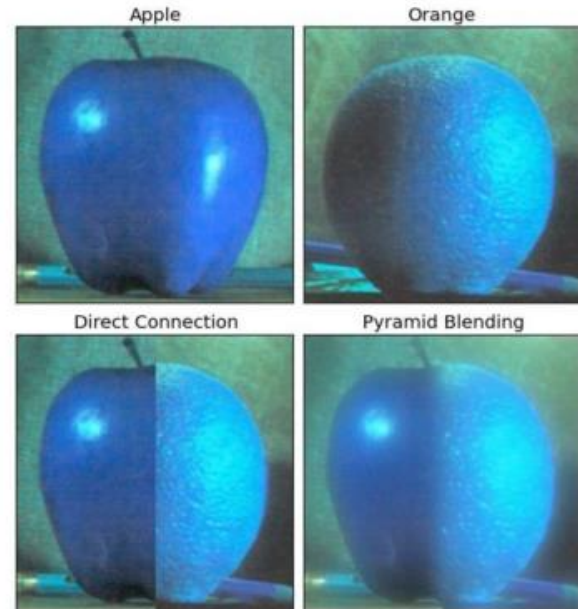
```
cv.imshow('Titulo Imagen', image3)
```

 OpenCV

Ojo con los detalles

OpenCV almacena las imágenes en BGR en lugar de RGB. Si leemos con *OpenCV* y visualizamos con *matplotlib* debemos tener cuidado.

```
import cv2 as cv
image3 = cv.imread(filename)
plt.imshow(image3)
plt.axis('off')
plt.show()
```



Ojo con los detalles

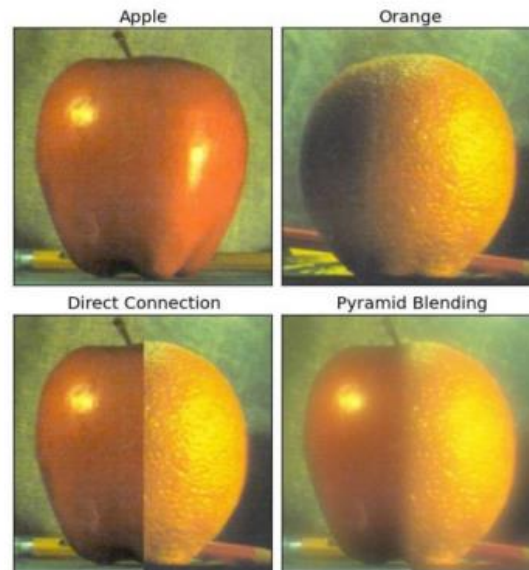
Debemos reordenar los canales para mostrarlos correctamente

Una alternativa es:

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

Otra alternativa es:

```
plt.imshow(image[:, :, ::-1])
```



Ojo con los detalles

cv2.imshow en Google Colab puede dar problemas



```
import cv2
```

```
img = cv2.imread('./drive/My Drive/Colab Notebooks/orapple.jpg')  
cv2.imshow('Imagen',img)
```



```
-----  
DisabledFunctionError                                Traceback (most recent call last)  
<ipython-input-2-ebc0249a3f51> in <module>()  
      2  
      3 img = cv2.imread('./drive/My Drive/Colab Notebooks/orapple.jpg')  
----> 4 cv2.imshow('Imagen',img)  
  
/usr/local/lib/python3.6/dist-packages/google/colab/_import_hooks/_cv2.py in wrapped(*args, **kwargs)  
     50 def wrapped(*args, **kwargs):  
     51     if not os.environ.get(env_var, False):  
--> 52         raise DisabledFunctionError(message, name or func.__name__)  
     53     return func(*args, **kwargs)  
     54
```

DisabledFunctionError: cv2.imshow() is disabled in Colab, because it causes Jupyter sessions to crash; see <https://github.com/jupyter/notebook/issues/3935>.

As a substitution, consider using

```
from google.colab.patches import cv2_imshow
```


Ojo con los detalles

cv2.imshow en Google Colab puede dar problemas

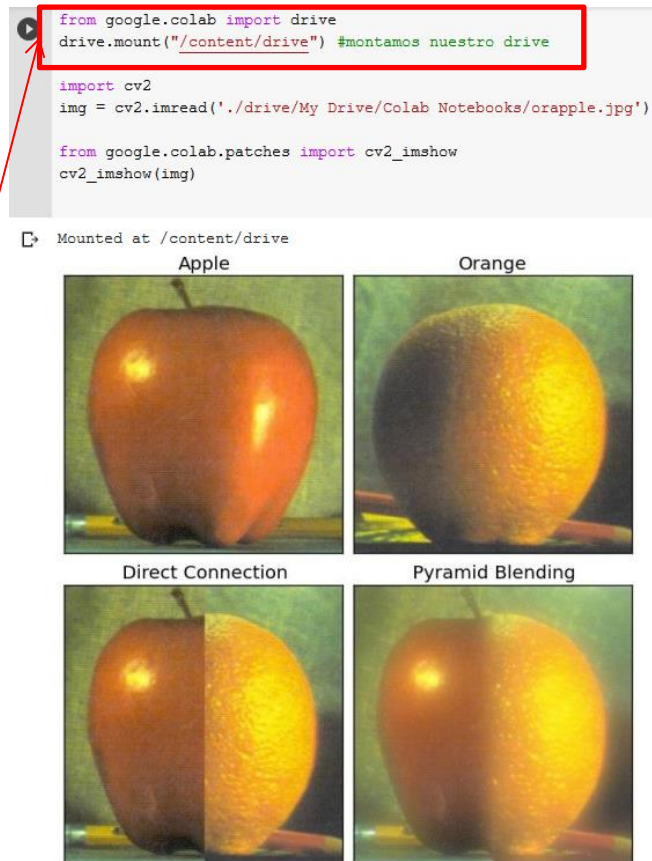
Una alternativa:

```
from google.colab.patches import cv2_imshow
cv2_imshow(img)
```

Otra alternativa:

```
import matplotlib.pyplot as plt
plt.imshow(img[:, :, ::-1])
```

Nota: acordaos de montar vuestro Drive en Colab para acceder a vuestros datos



Ojo con los detalles

- Debemos tener cuidado...
 - ¿por qué el código siguiente no saca un cuadrado blanco?

```
lado = 40
img = cv.imread('images/messi.jpg')
img2 = img.copy()
img2[img2.shape[0]-lado:img2.shape[0], 0:lado, :] = [1,1,1]
plt.imshow(img)
plt.imshow(img2)
```

La esquina superior
izquierda es el (0,0)

Filas de la imagen/matriz. Eje Y Columnas de la imagen/matriz. Eje X



Ojo con los detalles

- Porque OpenCV lee la imagen como una matriz de enteros (y 1, en el rango [0,255], es casi negro)

```
print(img.min())  
print(img.max())
```

```
0  
255
```

- Una forma de solucionarlo:

```
lado = 40
```

```
img = cv2.imread('images/messi.jpg') / 255
```

```
img2 = img.copy()
```

```
img2[img2.shape[0]-lado:img2.shape[0],0:lado,:] = [1,1,1]
```

```
plt.imshow(img2)
```



Ojo con los detalles

- Otro ejemplo:
 - Leemos/construimos una imagen con tipo uint8 [0-255], y le asignamos un valor intermedio (130, por decir algo → gris).
 - Matplotlib lo interpretará correctamente porque son valores enteros.
 - Si la convertimos a float64
 - Matplotlib espera que el rango de valores esté en [0-1]
 - Obtendremos el mensaje: `"WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers)."`
 - Y la imagen aparecerá saturada a blanco porque todos los valores superiores a 1 colapsarán en 1.

Ojo con los detalles

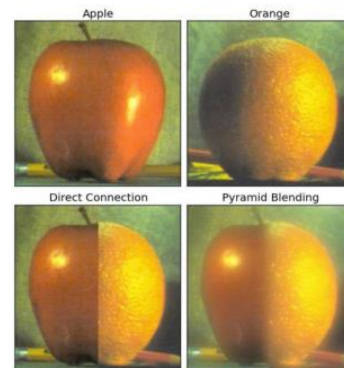
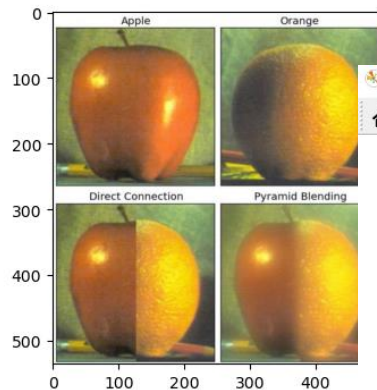
Si queréis mostrar varias figuras simultáneamente con *Matplotlib*, acordaos de usar `plt.figure()`, o se sobrescribirán (tanto en Spyder como en Colab)

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

```
filename = "images/orapple.jpg"
image = mpimg.imread(filename)
```

```
plt.figure(1)
plt.imshow(image)
```

```
plt.figure(2)
plt.axis("off")
plt.imshow(image)
```



Ojo con los detalles

Verificar cuidadosamente la documentación para evitar efectos/resultados indeseados. Por ejemplo, “`cv2.add()`” y el operador “`+`” no realizan exactamente la misma operación.

Véase

https://docs.opencv.org/4.12.0/d0/d86/tutorial_py_image_arithmetics.html

```
>>> x = np.uint8([[250]])
>>> y = np.uint8([[10]])
>>> print( cv.add(x,y) ) # 250+10 = 260 => 255
[[255]]
>>> print( x+y ) # 250+10 = 260 % 256 = 4
[[4]]
```

`add()` proporciona valores entre 0 y 255. Ese 260 es saturado al máximo valor (255).

`+` en este caso proporciona la suma de dos “8-bit unsigned integer (0 to 255)”. De modo que se “envuelve” módulo 256 automáticamente.

Recordatorio importante

- En visión por computador, los resultados que obtenemos se ven afectados por dos cuestiones principalmente:
 - **Visualización** (i.e., realizamos las operaciones correctas, pero nuestra visualización nos hace pensar que no es así)
 - **Operación** (i.e., no estamos realizando la operación adecuada (a nivel algorítmico o de implementación), pero nuestra visualización nos hace pensar que sí estamos haciendo lo correcto)
 - Esta última situación es más problemática, porque lo estamos haciendo mal y no nos damos cuenta...
- ¡Debemos ser conscientes de estas dos dimensiones en visión por computador!

Enlaces de interés

- OpenCV-Python:
 - https://docs.opencv.org/4.12.0/d6/d00/tutorial_py_root.html
- Google Colab:
 - <https://colab.research.google.com/notebooks/intro.ipynb>
 - https://colab.research.google.com/notebooks/basic_features_overview.ipynb
 - https://colab.research.google.com/drive/1RWGmqoEQdeyh5Tss_oGtsXsFk8hbLGtWp
- Jupyter Notebook:
 - <https://www.dataquest.io/blog/jupyter-notebook-tutorial/>